

CONTEXTGRAPH: REPAIR EFFECTS AS EXECUTABLE MEMORY

Anonymous authors

Paper under double-blind review

ABSTRACT

A past repair can identify what to observe in another program, but only if its learned relation survives the change of context. We investigate a memory representation that records a repair’s observable effect as an executable check, validated on the source before and after the repair. On a Matplotlib source repair, an automatically generated rendering check distinguishes the buggy and repaired behavior on two inputs. The central difficulty is turning this source contrast into an observation that informs the next repair decision. In our target pilot, supplying the check did not lead the solver to adapt and execute its relation. The evidence supports extracting a source repair effect; it does not yet establish a mechanism for using that effect in a new repair.

1 INTRODUCTION

Software repairs encode conditional knowledge. Adding a conversion, changing an ordering constraint, or copying an object can fix a bug because a particular input, runtime, and program state make that change necessary. Removing those conditions turns a useful experience into advice that may point in the wrong direction. Two issues can mention the same API and exception while requiring different edits. A memory that says what worked previously still leaves the agent to determine what is true now.

Our starting point is to ask what observation would make a past experience useful for the next repair decision. Does comparing the behavior before and after a source repair identify what is worth measuring in a new task? We study a memory representation that records this behavioral contrast, with verified target repairs as the measure of its value.

2 WHY RELEVANT EXPERIENCE CAN BE INSUFFICIENT

More guidance did not produce a clear gain. Our completed exploratory Related-Lite experiment compares six memory interfaces on the same 99 tasks from SWEContextBench (Zhu et al., 2026). The solver is SWE-agent 1.1.0 with a DeepSeek V4 Pro model alias, temperature zero, a 500-call limit, and a history processor retaining the last five observations. Flat retrieves historical patches; Card adds a repository procedure card; Graph, Rules, and File list expose other representations of the available experience. Related source fixes can closely match target repairs. Table 1 uses the benchmark’s frozen `resolved` criterion, which requires fail-to-pass success but does not veto every pass-to-pass regression.

Flat and Card each gain one observed repair over no memory; both exact paired tests give $p = 1$, providing no demonstrated gain in this setting.

Patch memory reaches all 99 targets in both conditions, and the expected procedure card reaches 98. However, reconstructing 198 Flat/Card histories shows that an original memory response is retained for a median of only five action prefixes. The median trajectories contain 497 and 495 actions, respectively. The development pilot below keeps memory persistent to remove this delivery constraint.

The same operation can be a repair or a bug. A Requests training example makes the applicability problem concrete. Issue 1724’s patch adds `method = builtin_str(method)`

Table 1: Earlier memory interventions on the same 99 Related-Lite tasks.

Memory interface	Resolved	Difference from no memory
No memory	17/99	—
Flat patches	18/99	+1.01 pp
Patches + procedure card	18/99	+1.01 pp
Graph	12/99	−5.05 pp
Rules	13/99	−4.04 pp
File list	9/99	−8.08 pp

to normalize an HTTP method in a Python 2 setting. Issue 2317 instead reports that this conversion turns Python 3 bytes `b'GET'` into the literal string `"b'GET'"`. Its training patch uses `to_native_string(method)`. A retrieved recommendation to perform the older conversion is related to the target but does not settle the correct action.

Here the issue already states the relevant input and runtime; rerunning that example would confirm known information. The useful check would instead distinguish actions whose applicability remains uncertain after reading the issue.

3 THE APPLICABILITY PILOT SUCCEEDS WITHOUT MEMORY

The completed pilot uses Requests 1921 and pytest 10051 from the training pool of our separate SWE-bench Verified split (Jimenez et al., 2024). The split allocates 350 tasks to memory construction and development and 150 to evaluation, with repository-stratified allocation and seed 42. The two pilot tasks belong to a 70-task development partition whose source rules are excluded from the remaining 280-source memory bank; the 150 evaluation tasks are unused here.

The two memory conditions receive the same persistent packet from ContextGraph’s dense, lexical, and graph retrieval pipeline. One uses the ordinary solver; the other adds an instruction to state an applicability condition, execute at most two short checks before editing, and use their observations to accept or reject a recommendation. The instruction does not enforce these decisions. We hold the model, tools, task images, \$3 metered threshold, and 100-call limit fixed, and grade each submitted patch immediately with the Verified harness.

Ordinary memory produces two empty patches after 39 and 49 calls. Adding the instruction produces two verified repairs after 36 and 17 calls. Removing only the memory packet also produces two verified repairs, after 30 and 34 calls; both patches are identical to their counterparts with memory. The ordinary runs never edit source, whereas both instruction conditions do. This two-task result shows that the observed repairs can occur without historical content. It directs method development toward the information supplied by experience, rather than expanding the instruction-only prototype.

4 REMEMBERING A REPAIR’S OBSERVABLE EFFECT

A check that passes on a repaired program may describe a general property without exposing anything learned from the repair. In a source-side trial on Matplotlib 23314, four checks generated from fixed code all pass before and after the repair: they exercise initialization, axis limits, sharing, and drawing order while missing the repaired visibility behavior. Source repair sensitivity cannot select an informative check from this pool.

Our revised construction gives the memory writer the source patch and fixed implementation and asks for a check of the changed observable behavior. It executes the check on the buggy and repaired source with two legal inputs. An admitted check must run normally in both versions and expose a false-to-true relation. The stored memory contains the source issue, relation, and executable check.

The observable effect matters because a repair can also change incidental internal state. A generated check of Matplotlib’s projection cache detects the source change, but the corresponding cache does not exist in the two-dimensional target API. Changing both the operation and the measured state

would invent a different check. We therefore require observations of rendered output or documented API results, rather than an implementation-specific cache. A generated output check renders a hidden blue axes region on a red figure. Before the source repair, the two inputs leave 57,600 and 40,000 non-red pixels; after the repair, both leave zero.

Using this contrast in a new repair requires binding the check to the target’s objects and API while preserving its measured relation. Source validity alone does not establish this binding or show that the resulting observation changes a repair decision.

On training target Matplotlib 20826, we compared providing the original source patch, the effect check, and no memory under the same applicability instruction, \$3 metered threshold, and 100-call limit. The three arms used 32, 34, and 33 calls, respectively. All stopped at the cost threshold without editing production code, submitted empty patches, and were immediately recorded as unresolved by the official verifier. They reproduced the target’s two-dimensional axis-clearing fault, but the effect-check arm never adapted and executed the source rendering relation. This comparison tests supplying a source check as text, not an executed target-transfer mechanism. We stopped expanding this text-delivery prototype.

5 RELATED WORK

Experience and skills. ExpeL distills reusable insights from agent trajectories (Zhao et al., 2024). SWE-Exp uses historical experience for software issue resolution (Chen et al., 2025), and MemGov-ern turns human experience into searchable cards (Wang et al., 2026). CODESKILL learns a skill extraction and maintenance policy using feedback from a frozen downstream solver (Li et al., 2026). ExpeRepair retrieves historical test demonstrations and reflective insights to generate reproduction and boundary tests (Mu et al., 2026).

Repair patterns and context. Getafix learns fix patterns from human patches and uses surrounding code to rank applicable edits (Bader et al., 2019). VulDebugger compares expected constraints with observed runtime state to guide vulnerability repair (Liu et al., 2025). MemRepair combines historical fixes, security patterns, and refinement trajectories with runtime feedback (Liu et al., 2026). MR-Scout parameterizes relations from existing tests, and MR-Coupler generates relation tests from coupled methods and evaluates historical defects (Xu et al., 2023; 2026). The question here is whether a source repair’s observable effect can inform an executable check for a different repair task.

Graph retrieval and working memory. HippoRAG motivates associative retrieval with Personalized PageRank (Gutiérrez et al., 2024). SWE-MeM studies working-memory management (Gao et al., 2026), while cue-anchored memory studies delivery through coding-task and compaction experiments (Saha, 2026). AdaMEM retrieves experience at intermediate states and synthesizes an updated strategy (Zhang et al., 2026). Decision-Aware Memory Cards scores context by estimated effects on actions (Guan et al., 2026).

AI USE STATEMENT

Generative AI assisted literature review, method development, implementation, experiments, analysis, and writing. The manuscript is a working draft and has not been submitted.

REFERENCES

- Johannes Bader, Andrew Scott, Michael Pradel, and Satish Chandra. Getafix: Learning to fix bugs automatically, 2019. URL <https://arxiv.org/abs/1902.06111>.
- Silin Chen, Shaoxin Lin, Yuling Shi, Heng Lian, Xiaodong Gu, Longfei Yun, Dong Chen, Lin Cao, Jiyang Liu, Nu Xia, and Qianxiang Wang. SWE-Exp: Experience-driven software issue resolution, 2025. URL <https://arxiv.org/abs/2507.23361>.

- Shuzheng Gao, Wenhao Zeng, Zhaojian Yu, Jianqiao Wangni, Chaozheng Wang, Kai Cai, Shilin He, and Michael R. Lyu. Swe-mem: Learning adaptive memory management for long-horizon coding agents, 2026. URL <https://arxiv.org/abs/2606.28434v1>. Version 1.
- Xinyu Guan, Qianyang Zhao, and Yuming Deng. Decision-aware memory cards: Counterfactual-inspired context selection and compression for tool-using LLM agents, 2026. URL <https://arxiv.org/abs/2606.08151v1>.
- Bernal Jiménez Gutiérrez, Yiheng Shu, Yu Gu, Michihiro Yasunaga, and Yu Su. Hipporag: Neurobiologically inspired long-term memory for large language models, 2024. URL <https://arxiv.org/abs/2405.14831v3>.
- Carlos E. Jimenez, John Yang, Alexander Wettig, Shunyu Yao, Kexin Pei, Ofir Press, and Karthik Narasimhan. Swe-bench: Can language models resolve real-world github issues?, 2024. URL <https://arxiv.org/abs/2310.06770>.
- Yanzhou Li, Yiran Zhang, Xiaoyu Zhang, Xiaoxia Liu, and Yang Liu. Codeskill: Learning self-evolving skills for coding agents, 2026. URL <https://arxiv.org/abs/2605.25430v1>. Version 1.
- Simiao Liu, Li Zhang, Fang Liu, Xiaoli Lian, Yang Liu, and Yinghao Zhu. MemRepair: Hierarchical memory for agentic repository-level vulnerability repair, 2026. URL <https://arxiv.org/abs/2605.17444v1>.
- Zhengyao Liu, Yunlong Ma, Jingxuan Xu, Junchen Ai, Xiang Gao, Hailong Sun, and Abhik Roychoudhury. Agent that debugs: Dynamic state-guided vulnerability repair, 2025. URL <https://arxiv.org/abs/2504.07634>.
- Fangwen Mu, Junjie Wang, Lin Shi, Song Wang, Shoubin Li, and Qing Wang. ExpeRepair: Dual-memory enhanced LLM-based repository-level program repair, 2026. URL <https://arxiv.org/abs/2506.10484v2>. Version 2; accepted at FSE 2026.
- Swapnanil Saha. Delivery, not storage: Cue-anchored working memory as a harness property for coding agents, 2026. URL <https://arxiv.org/abs/2607.20972v1>. Version 1.
- Qihao Wang, Ziming Cheng, Shuo Zhang, Fan Liu, Rui Xu, et al. MemGovern: Enhancing code agents through learning from governed human experiences, 2026. URL <https://arxiv.org/abs/2601.06789v2>.
- Congying Xu, Valerio Terragni, Hengcheng Zhu, Jiarong Wu, and Shing-Chi Cheung. MR-Scout: Automated synthesis of metamorphic relations from existing test cases, 2023. URL <https://arxiv.org/abs/2304.07548>.
- Congying Xu, Hengcheng Zhu, Songqiang Chen, Jiarong Wu, Valerio Terragni, and Shing-Chi Cheung. MR-Coupler: Automated metamorphic test generation via functional coupling analysis, 2026. URL <https://arxiv.org/abs/2604.10126>.
- Yunxiang Zhang, Yiheng Li, Ali Payani, and Lu Wang. AdaMEM: Test-time adaptive memory for language agents, 2026. URL <https://arxiv.org/abs/2606.05684v1>.
- Andrew Zhao, Daniel Huang, Quentin Xu, Matthieu Lin, Yong-Jin Liu, and Gao Huang. Expel: Llm agents are experiential learners, 2024. URL <https://arxiv.org/abs/2308.10144v3>.
- Jiayuan Zhu, Junde Wu, Minhao Hu, Shengda Zhu, Jiazhen Pan, Weixiang Shen, Yijun Yang, Fenglin Liu, Jianye Hao, Yueming Jin, Qirong Ho, and Min Xu. Swe context bench: A benchmark for context learning in coding, 2026. URL <https://arxiv.org/abs/2602.08316v3>. Version 3.